

纯虚函数与抽象类

1.抽象类

问题：如何用一个函数接受不同类型的类？

2.纯虚函数

纯虚函数是没给出实现的虚函数，函数体用“=0”表示，如：

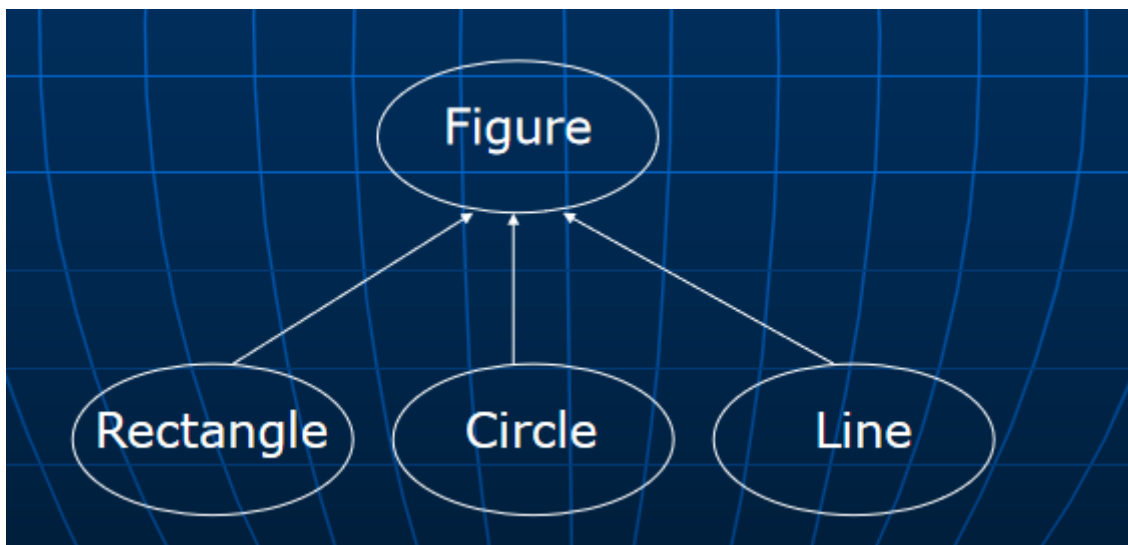
```
1 class A
2 { .....
3     public:
4         virtual int f()=0; //纯虚函数
5     .....
6 };
```

纯虚函数需要在派生类中给出实现。

包含纯虚函数的类称为抽象类。

抽象类不能用于创建对象。

抽象类的作用是为派生类提供一个基本框架和一个公共的对外接口。



```
1 class Figure //抽象基类
2 { public:
3     virtual void draw() const=0; //派生类共有的功能
4     virtual void input_data()=0; //派生类共有的功能
5 };
6 class Rectangle: public Figure
7 {     double left,top,right,bottom;
8     public:
9         void draw() const
10        { ..... //画矩形
11        }
12        void input_data()
13        {     cout << "请输入矩形的左上角和右下角坐标 (x1,y1,x2,y2) : ";
14            cin >> left >> top >> right >> bottom;
```

```

15     }
16     double area() const //派生类特有的功能
17     { return (bottom-top)*(right-left);
18     }
19     .....
20 };
21
22 const double PI=3.1416;
23 class Circle: public Figure
24 {     double x,y,r;
25     public:
26     void draw() const
27     {     ..... //画圆
28     }
29     void input_data()
30     {     cout << "请输入圆的圆心坐标和半径 (x,y,r) : ";
31         cin >> x >> y >> r;
32     }
33     double area() const { return r*r*PI; }
34     .....
35 };
36
37 class Line: public Figure
38 {     double x1,y1,x2,y2;
39     public:
40     void draw() const
41     {     ..... //画线
42     }
43     void input_data()
44     {     cout << "请输入线段的起点和终点坐标 (x1,y1,x2,y2) : ";
45         cin >> x1 >> y1 >> x2 >> y2;
46     }
47     .....
48 };
49 .....
50 const int MAX_NUM_OF_FIGURES=100;
51 Figure *figures[MAX_NUM_OF_FIGURES];
52 int count=0;
53
54 图形数据的输入:
55 for (count=0; count<MAX_NUM_OF_FIGURES; count++)
56 {     int shape;
57     do
58     {     cout << "请输入图形的种类(0: 线段, 1: 矩形, 2: 圆, -1: 结束): ";
59         cin >> shape;
60     } while (shape < -1 || shape > 2);
61     if (shape == -1) break;
62     switch (shape)
63     {     case 0: //线
64         figures[count] = new Line; break;
65         case 1: //矩形
66         figures[count] = new Rectangle; break;
67         case 2: //圆
68         figures[count] = new Circle; break;
69     }
70     figures[count]->input_data(); //动态绑定到相应类的input_data

```

```

71 }
72
73 图形的输出:
74 for (int i=0; i<count; i++)
75     figures[i]->draw();
76     //通过动态绑定调用相应类的draw

```

3.例：用抽象类实现抽象数据结构类型Stack

抽象数据类型（abstract data type）是指只考虑类型的抽象性质，而不考虑该类型的实现。

```

1  class Stack
2  {   public:
3      virtual void push(int i)=0;
4      virtual void pop(int& i)=0;
5  };
6
7  class ArrayStack: public Stack
8  {   int elements[100],top;
9      public:
10     ArrayStack() { top = -1; }
11     void push(int i) { ..... }
12     void pop(int& i) { ..... }
13 };
14 class LinkedStack: public Stack
15 {   struct Node
16     {   int content;
17         Node *next;
18     } *first;
19     public:
20     LinkedStack() { first = NULL; }
21     void push(int i) { ..... }
22     void pop(int& i) { ..... }
23 };
24
25 void f(Stack &st)
26 {   .....
27     st.push(...); //将根据st引用的对象类来确定push的归属
28     .....
29     st.pop(...); //将根据st引用的对象类来确定pop的归属
30     .....
31 }
32 int main()
33 {   ArrayStack st1;
34     LinkedStack st2;
35     f(st1); //OK
36     f(st2); //OK
37     .....
38 }
39

```

抽象类可以实现类的真正抽象作用。使用者无法见到类的具体定义和实现细节。

